FastMCP Observability Project Guide

Simple file-by-file explanation of the callback + skills implementation

Project: fastmcp_observability_callback_simple

Purpose: Explain every file, class, function, and method in the simplified FastAPI + Deep Agents + FastMCP observability project in plain English.

1. Big Picture

The project is a small chatbot backend. A user sends a message to FastAPI, the backend sends it to a Deep Agent, the agent may use MCP tools, and the app records what happened using OpenTelemetry spans plus a local JSONL metrics file.

```
Client -> FastAPI -> SessionStore -> AgentEngine -> Deep Agent + Skills -> MCPRuntime -> Core
Gateway -> MCP Tool Servers
```

The most important observability idea: RequestObserver uses LangChain callback events, so LLM calls and tool calls are based on real runtime events, not estimates from message history.

2. What Happens During One /chat Request

Step	Plain-English explanation
1. App startup	backend.main.create_app loads Settings, configures OpenTelemetry, starts MCPRuntime, starts AgentEngine, and creates ChatSessionStore plus JSONLMetricsWriter.
2. Tools load once	AgentEngine.start asks MCPRuntime for all tools. MCPRuntime connects to the core gateway, loads tools, deduplicates them by name, and returns them.
3. Agent is created	AgentEngine.start calls create_deep_agent with model, tools, system prompt, and skills=[settings.skills_dir].
4. Chat request arrives	POST /chat validates ChatRequest, records request-level span attributes, and calls ChatSessionStore.chat.
5. Session locks	ChatSessionStore finds or creates the session, locks it, and calls AgentEngine.respond.
6. Agent runs	AgentEngine.respond creates RequestObserver and invokes the Deep Agent with callbacks=[observer].
7. Real events are recorded	RequestObserver receives LangChain callbacks for model starts and tool starts/ends/errors.
8. Metrics are returned	AgentEngine reads observer.llm_calls, observer.tools_used(), and observer.tool_summary().
9. State is saved	ChatSessionStore saves the updated message state back into the session.
10. Final telemetry is written	backend.main writes one ChatTelemetryEvent row to metrics/chat_metrics.jsonl and returns ChatResponse.

3. File Map

File	Main purpose

.env.example	Example environment configuration for the app.
README.md	Human-facing overview and run instructions.
pyproject.toml	Python package metadata and dependency list.
backend/__init__.py	Marks backend as a Python package.
backend/config.py	Loads typed runtime settings from .env and environment variables.
backend/main.py	FastAPI application entry point.
backend/schemas.py	Pydantic models for the HTTP API.
backend/agent.py	Creates and runs the shared Deep Agent.
backend/session_store.py	Keeps in-memory conversation state by session_id.
backend/mcp_runtime.py	Connects the backend to MCP gateways and loads tools.
backend/observability/__init__.py	Marks observability as a Python package.
backend/observability/otel.py	Small OpenTelemetry setup helpers.
backend/observability/models.py	Pydantic models for observability data.
backend/observability/request_observer.py	Main request-level observability hook for LangChain/Deep Agents execution.
backend/observability/metrics.py	Writes local JSONL metrics.
gateways/__init__.py	Marks gateways as a Python package.
gateways/core_gateway.py	FastMCP gateway that mounts built-in MCP servers.
mcp_server/__init__.py	Marks mcp_server as a Python package.
mcp_server/calculator_server.py	Small FastMCP server exposing calculator tools.
mcp_server/quote_counter_server.py	Small FastMCP server for quoted text counting.
skills/math/SKILL.md	Deep Agents native skill for math requests.
skills/quote_count/SKILL.md	Deep Agents native skill for quoted text counting requests.

4. File-by-File Details

.env.example

What it does: Example environment configuration for the app.

How it fits: Defines the LLM connection, CORS origins, metrics path, MCP gateway URLs, OpenTelemetry settings, and the skills folder path.

Class / function / method / item	Simple explanation
LLM_* variables	Tell the backend which OpenAI-compatible model endpoint to call.
SKILLS_DIR	Points Deep Agents to the skills folder. This is always used; it is not an enable/disable switch.
CORE_GATEWAY_URL	The MCP gateway URL that the backend connects to when loading tools.
OTEL_* variables	Control whether OpenTelemetry spans are off, printed to the console, or exported to a collector.
MCP_* and gateway variables	Control where the MCP servers and gateway run locally.

README.md

What it does: Human-facing overview and run instructions.

How it fits: Explains the architecture, observability flow, skills layout, install steps, run commands, and how to view metrics.

Class / function / method / item	Simple explanation

Architecture section	Shows the main runtime path from API request to MCP tools.
Observability flow section	Explains that RequestObserver records real LangChain model/tool events.
Skills section	Explains that Deep Agents native skills replace the older custom skills registry.
Run and test sections	Give the four-terminal startup process and curl examples.

pyproject.toml

What it does: Python package metadata and dependency list.

How it fits: Lets the project install with pip install -e . and declares required libraries like FastAPI, Deep Agents, FastMCP, LangChain, Pydantic, and OpenTelemetry.

Class / function / method / item	Simple explanation
[project]	Defines project name, version, Python requirement, and dependencies.
[project.optional-dependencies]	Defines dev tools such as ruff and pytest.
[tool.hatch.build.targets.wheel]	Tells the package builder which Python packages belong in the project.

backend/__init__.py

What it does: Marks backend as a Python package.

How it fits: Allows imports like from backend.agent import AgentEngine.

backend/config.py

What it does: Loads typed runtime settings from .env and environment variables.

How it fits: This is where raw strings from the environment become a Pydantic Settings object used by the rest of the app.

Class / function / method / item	Simple explanation
_csv(value)	Splits a comma-separated string into a clean list. Used for CORS origins.
_bool(value, default)	Converts environment strings like true, yes, on, or 1 into a real boolean.
Settings	Pydantic model containing all app configuration.
Settings.from_env()	Loads .env, reads environment variables, applies defaults, and returns a Settings object.

backend/main.py

What it does: FastAPI application entry point.

How it fits: Creates the app, configures OpenTelemetry, starts the MCP runtime and agent engine, exposes /health and /chat, and writes final request metrics.

Class / function / method / item	Simple explanation
create_app(settings=None)	Builds the FastAPI app and wires all runtime dependencies together.
lifespan(app)	Startup/shutdown context inside create_app. Starts MCPRuntime and AgentEngine, stores them on app.state, and stops the runtime on shutdown.

health()	Simple GET /health endpoint returning {"status": "ok"}.
chat(req)	POST /chat endpoint. Runs one chat turn, records request-level span attributes, writes one JSONL telemetry row, and returns ChatResponse.
app = create_app()	Module-level app object used by uvicorn backend.main:app.

backend/schemas.py

What it does: Pydantic models for the HTTP API.

How it fits: Defines the shape of requests and responses for /chat.

Class / function / method / item	Simple explanation
ChatRequest	Input model for /chat. Requires session_id and message.
ChatResponseMetrics	Metrics returned to the caller, including total time, engine time, LLM calls, tool calls, and lock wait time.
ChatResponse	Output model for /chat. Includes reply, tools_used, mode, trace_id, and metrics.

backend/agent.py

What it does: Creates and runs the shared Deep Agent.

How it fits: This is the center of the agent execution path. It loads tools, creates the agent with native skills, runs a chat turn, and collects metrics from RequestObserver.

Class / function / method / item	Simple explanation
DEFAULT_SYSTEM_PROMPT	Fallback system prompt if APP_SYSTEM_PROMPT is not set.
EngineResult	Pydantic result returned from the engine to the session layer.
AgentEngine.__init__(settings, mcp_runtime)	Stores settings and MCP runtime. The actual agent is created later during startup.
AgentEngine.start()	Loads all MCP tools, records startup span attributes, and creates the Deep Agent with model, tools, system prompt, and skills.
AgentEngine.respond(session_id, state, user_text)	Runs one chat turn. Builds the agent input, creates RequestObserver, invokes the agent, extracts the final AI reply, builds EngineMetrics, and returns updated state.
AgentEngine._last_ai_message_text(messages)	Finds the last AIMessage and returns its text for the API response.

backend/session_store.py

What it does: Keeps in-memory conversation state by session_id.

How it fits: Prevents two requests for the same session from changing the same message history at the same time.

Class / function / method / item	Simple explanation
ChatSession	Pydantic model holding one session state dictionary and one asyncio lock.
ChatResult	Pydantic result returned from the session layer to the API layer.
ChatSessionStore.__init__(engine)	Stores the AgentEngine and initializes the in-memory session dictionary.

ChatSessionStore.chat(session_id, message)	Gets/creates the session, starts a session.chat span, locks the session, calls AgentEngine.respond, updates session state, and returns ChatResult.
ChatSessionStore._get_or_create_session(session_id)	Creates a new ChatSession if one does not exist, otherwise returns the existing one.

backend/mcp_runtime.py

What it does: Connects the backend to MCP gateways and loads tools.

How it fits: Keeps MCP connection/session logic separate from the agent and observability logic.

Class / function / method / item	Simple explanation
MCPRuntime.__init__(settings)	Stores settings and initializes empty connection/session/tool caches.
MCPRuntime.start()	Builds the gateway connection map, currently including the core gateway and optionally a BYOT gateway URL.
MCPRuntime.get_all_tools()	Loads tools from each gateway, deduplicates them by name, records tool count/name span attributes, and returns the tool list to the agent.
MCPRuntime._load_tools_for_gateway(alias)	Opens one MCP gateway session and caches its loaded tools.
MCPRuntime._deduplicate_tools_by_name()	Returns one tool per name while keeping the original load order.
MCPRuntime._join_tool_names(tools)	Turns a tool list into a comma-separated tool name string for spans/logging.
MCPRuntime.stop()	Closes opened MCP sessions and clears runtime caches during app shutdown.

backend/observability/__init__.py

What it does: Marks observability as a Python package.

How it fits: Allows imports like from backend.observability.otel import setup_otel.

backend/observability/otel.py

What it does: Small OpenTelemetry setup helpers.

How it fits: Centralizes tracer setup so the rest of the app only asks for tracers and span IDs.

Class / function / method / item	Simple explanation
setup_otel(enabled, service_name, otlp_endpoint)	Configures the global tracer provider once. If an OTLP endpoint is provided, spans export there; otherwise they print to the console.
instrument_fastapi(app, enabled)	Adds automatic HTTP request spans to the FastAPI app when tracing is enabled.
get_tracer(name)	Returns an OpenTelemetry tracer for a module.
current_trace_id()	Returns the current trace ID as a hex string, or None if there is no valid active span.

backend/observability/models.py

What it does: Pydantic models for observability data.

How it fits: Gives metrics and telemetry a consistent shape before they are attached to spans, returned by the API, or written to JSONL.

Class / function / method / item	Simple explanation
ToolCallStarted	Temporary record stored while a tool is still running.
ToolCallRecord	Final record for one completed or failed tool execution.
ToolSummary	Rollup of all tool executions in one chat request.
ToolSummary.to_span_attributes()	Converts tool summary fields into OpenTelemetry span attributes.
EngineMetrics	Metrics produced by the engine for one chat turn.
ChatTelemetryEvent	One final telemetry event written after a /chat request completes or fails.
ChatTelemetryEvent.to_jsonl_row()	Flattens the telemetry event into a JSON-serializable dictionary for JSONL output.

backend/observability/request_observer.py

What it does: Main request-level observability hook for LangChain/Deep Agents execution.

How it fits: This is the source of truth for real LLM and tool activity during an agent request. It uses LangChain callback events instead of estimating from message history.

Class / function / method / item	Simple explanation
RequestObserver.__init__()	Initializes counters and dictionaries used to track one request.
on_chat_model_start(...)	LangChain callback fired when a chat model call starts. Increments the real LLM call count.
on_llm_start(...)	LangChain callback fired when a non-chat LLM call starts. Also increments the real LLM call count.
on_tool_start(...)	LangChain callback fired when a tool begins. Stores start time and opens a tool.invoke span.
on_tool_end(...)	LangChain callback fired when a tool succeeds. Records duration/output size, saves ToolCallRecord, and ends the tool span.
on_tool_error(...)	LangChain callback fired when a tool fails. Records error details, saves ToolCallRecord, records exception on the span, and ends the span.
tools_used()	Returns the tool names that actually ran during this request.
tool_summary()	Builds a ToolSummary from the completed tool records.
add_tool_events_to_span(span)	Adds a compact tool.completed event timeline to the parent engine span.
_record_llm_start(run_id, model_type)	Deduplicates model-start callback events and increments llm_calls only once per LangChain run ID.
_duration_ms(start_time_s)	Converts a perf_counter start time into elapsed milliseconds.

backend/observability/metrics.py

What it does: Writes local JSONL metrics.

How it fits: Provides a simple local metrics file even when OpenTelemetry export is disabled.

Class / function / method / item	Simple explanation
JSONLMetricsWriter.__init__(path)	Creates the metrics directory if needed and prepares a thread lock.
JSONLMetricsWriter.write(event)	Adds a UTC timestamp and appends one telemetry row to the JSONL file.

gateways/__init__.py

What it does: Marks gateways as a Python package.

How it fits: Keeps gateway code importable and packaged.

gateways/core_gateway.py

What it does: FastMCP gateway that mounts built-in MCP servers.

How it fits: The backend connects to this one gateway URL instead of directly connecting to each MCP server.

Class / function / method / item	Simple explanation
build_gateway()	Creates the gateway and mounts calculator tools under math and quote tools under quote.
__main__ block	Runs the gateway over HTTP using host, port, and path environment variables.

mcp_server/__init__.py

What it does: Marks mcp_server as a Python package.

How it fits: Keeps the example MCP servers importable and packaged.

mcp_server/calculator_server.py

What it does: Small FastMCP server exposing calculator tools.

How it fits: Provides simple tools that demonstrate the agent -> gateway -> MCP server flow.

Class / function / method / item	Simple explanation
mcp = FastMCP("Calculator")	Creates the calculator MCP server.
add(a, b)	MCP tool that adds two numbers and returns an int if the result is a whole number.
subtract(a, b)	MCP tool that subtracts b from a and returns an int if the result is a whole number.
__main__ block	Runs the calculator MCP server over HTTP.

mcp_server/quote_counter_server.py

What it does: Small FastMCP server for quoted text counting.

How it fits: Provides a text-processing tool that demonstrates non-math tool usage.

Class / function / method / item	Simple explanation
mcp = FastMCP("Quote Counter")	Creates the quote counter MCP server.
_first_quoted_segment(text)	Finds the first double-quoted or single-quoted segment in a string.
count_quoted_chars(text)	MCP tool that returns the quoted text and its character count.
__main__ block	Runs the quote counter MCP server over HTTP.

skills/math/SKILL.md

What it does: Deep Agents native skill for math requests.

How it fits: Gives the agent extra instructions about using calculator tools for exact arithmetic.

Class / function / method / item	Simple explanation
frontmatter name/description	Tells Deep Agents when the skill is relevant.
skill body	Instructs the agent to prefer tools for exact arithmetic and answer clearly.

skills/quote_count/SKILL.md

What it does: Deep Agents native skill for quoted text counting requests.

How it fits: Gives the agent extra instructions about when and how to use the quote-counting tool.

Class / function / method / item	Simple explanation
frontmatter name/description	Tells Deep Agents when the skill is relevant.
skill body	Clarifies what should be counted and how to explain the result.

5. Observability Pieces in Simple Terms

Concept	Simple meaning
OpenTelemetry spans	A timeline of work. The app creates spans such as engine.respond, engine.agent_invoke, and tool.invoke add.
RequestObserver	The object that watches one agent request. LangChain calls its methods when real model and tool events happen.
EngineMetrics	The structured metrics returned from the engine, including llm_calls and tool timing counts.
ChatTelemetryEvent	The final row written after each /chat request.
metrics/chat_metrics.jsonl	A simple local file where each line is one completed or failed chat request.
trace_id	The ID that connects API output, spans, and telemetry rows when tracing is enabled.

6. Skills in This Version

Skills are always part of the app. There is no custom skills registry and no skills_enabled switch. The agent is created with skills=[settings.skills_dir], so Deep Agents handles skill discovery and loading from SKILL.md files.

Skill file	What it teaches the agent
skills/math/SKILL.md	Use calculator tools for exact arithmetic and answer briefly.
skills/quote_count/SKILL.md	Use the quote counting tool when the user asks about quoted text or characters inside quotes.

7. Mental Model for Developers

- backend/main.py owns the web API and final request telemetry.
- backend/session_store.py owns conversation memory and session locking.
- backend/agent.py owns the Deep Agent and converts one user turn into an EngineResult.
- backend/mcp_runtime.py owns MCP gateway connections and tool loading.

- backend/observability/request_observer.py owns real LLM/tool event collection.
- backend/observability/models.py owns the typed telemetry data shapes.
- mcp_server/* owns the actual tools.
- gateways/core_gateway.py exposes the tool servers through one MCP gateway.
- skills/*/SKILL.md gives Deep Agents extra instructions without custom registry code.

Rule of thumb: If you want to understand a request, read main.py -> session_store.py -> agent.py -> request_observer.py. If you want to understand tools, read mcp_runtime.py -> core_gateway.py -> mcp_server/*.py.